

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

ENTREGA 3B

Header, Logout y Perfil

Cierre de la Fase 1 — Fundación

Proyecto: gestión-académica

Prerrequisito: Entregas 1A, 1B, 2A, 2B y 3A completadas

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Implementar un header con dropdown de usuario que incluye avatar, nombre y opción de logout.
- Crear la funcionalidad de cierre de sesión completa con redirección.
- Construir un custom hook (useUsuario) para acceder a los datos del usuario desde cualquier Client Component.
- Crear una API Route para actualizar el perfil del usuario.
- Completar las páginas placeholder para todas las secciones del dashboard.

Prerrequisitos

- ✓ Entregas 1A a 3A completadas
- ✓ Sidebar dinámico por rol funcionando
- ✓ Al menos un usuario de cada rol creado para pruebas

Crear el hook useUsuario

Un custom hook nos permite encapsular la lógica de obtener los datos del usuario autenticado en un solo lugar reutilizable. Cualquier Client Component podrá importar este hook para acceder al nombre, rol y email del usuario.

Crea el archivo **hooks/use-usuario.ts**:

```
Archivo: hooks/use-usuario.ts
"use client"

import { useEffect, useState } from "react"
import { createClient } from "@/lib/supabase/client"

type Usuario = {
  id: string
  email: string
  nombre_completo: string
  rol: string
  carrera: string | null
  telefono: string | null
  avatar_url: string | null
}
```

```

export function useUsuario() {
  const [usuario, setUsuario] = useState<Usuario | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    async function fetchUsuario() {
      try {
        const supabase = createClient()

        // 1. Obtener el usuario autenticado
        const {
          data: { user },
        } = await supabase.auth.getUser()

        if (!user) {
          setLoading(false)
          return
        }

        // 2. Obtener el perfil de la BD
        const { data: perfil } = await supabase
          .from("usuarios_perfil")
          .select("*")
          .eq("id", user.id)
          .single()

        if (perfil) {
          setUsuario({
            id: user.id,
            email: user.email || "",
            nombre_completo: perfil.nombre_completo,
            rol: perfil.rol,
            carrera: perfil.carrera,
            telefono: perfil.telefono,
            avatar_url: perfil.avatar_url,
          })
        }
      } catch (error) {
        console.error("Error al obtener usuario:", error)
      } finally {
        setLoading(false)
      }
    }

    fetchUsuario()
  }, [])

  return { usuario, loading }
}

```

¿Qué es un Custom Hook?

Un custom hook es una función que empieza con **use** y encapsula lógica reutilizable con otros hooks de React (useState, useEffect, etc.). Ventajas:

- **Reutilizable:** cualquier componente puede llamar useUsuario() sin duplicar código.
- **Encapsulado:** la lógica de Supabase queda oculta. El componente solo recibe { usuario, loading }.
- **Testeable:** puedes probar el hook de forma aislada.

Nota: ¿Por qué no usamos este hook en el layout?

El layout del dashboard es un Server Component que obtiene los datos directamente de Supabase con await. El hook useUsuario es para Client Components que no pueden hacer await directamente. Dos mecanismos diferentes para el mismo dato, cada uno optimizado para su contexto.

Crear el componente Header con dropdown de usuario

Vamos a reemplazar el header simple del layout con un componente más completo que incluya el avatar del usuario, su nombre y un menú desplegable con opciones.

Crea el archivo **components/dashboard/header.tsx**:

```
Archivo: components/dashboard/header.tsx
"use client"

import { useRouter } from "next/navigation"
import { Logout, UserCircle } from "lucide-react"
import { createClient } from "@lib/supabase/client"

import { Avatar, AvatarFallback } from "@components/ui/avatar"
import { Button } from "@components/ui/button"
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu"

interface HeaderProps {
  nombreUsuario: string
  email: string
  rol: string
}

export function DashboardHeader({
  nombreUsuario,
```

```

    email,
    rol,
  }: HeaderProps) {
    const router = useRouter()

    // Obtener las iniciales del nombre para el avatar
    function getInitials(name: string): string {
      return name
        .split(" ")
        .map((word) => word.charAt(0))
        .slice(0, 2)
        .join("")
        .toUpperCase()
    }

    async function handleLogout() {
      const supabase = createClient()
      await supabase.auth.signOut()
      router.push("/login")
      router.refresh()
    }

    return (
      <div className="ml-auto flex items-center gap-4">
        {/* — Dropdown del usuario — */}
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="ghost" className="relative h-9 w-9 rounded-full">
              <Avatar className="h-9 w-9">
                <AvatarFallback className="bg-primary text-primary-foreground text-sm">
                  {getInitials(nombreUsuario)}
                </AvatarFallback>
              </Avatar>
            </Button>
          </DropdownMenuTrigger>

          <DropdownMenuContent className="w-56" align="end" forceMount>
            {/* — Información del usuario — */}
            <DropdownMenuLabel className="font-normal">
              <div className="flex flex-col space-y-1">
                <p className="text-sm font-medium leading-none">
                  {nombreUsuario}
                </p>
                <p className="text-xs leading-none text-muted-foreground">
                  {email}
                </p>
                <p className="text-xs leading-none text-muted-foreground capitalize">
                  {rol}
                </p>
              </div>
            </DropdownMenuLabel>
          </DropdownMenuContent>
        </DropdownMenu>
      </div>
    )
  }
}

```

```

    <DropdownMenuSeparator />

    { /* — Opciones del menú — */ }
    <DropdownMenuItem
      onClick={() => router.push(`/${rol}/perfil`)}
      className="cursor-pointer"
    >
      <UserCircle className="mr-2 h-4 w-4" />
      Mi Perfil
    </DropdownMenuItem>

    <DropdownMenuSeparator />

    <DropdownMenuItem
      onClick={handleLogout}
      className="cursor-pointer text-destructive focus:text-destructive"
    >
      <LogOut className="mr-2 h-4 w-4" />
      Cerrar Sesión
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>
</div>
)
}

```

Desglose del Header

getInitials(): Extrae las primeras letras de cada palabra del nombre. "Juan Pérez" se convierte en "JP". Se usa como fallback del avatar cuando no hay foto.

handleLogout(): Llama a `supabase.auth.signOut()` que elimina las cookies de sesión del navegador. Luego redirige a `/login` y hace `refresh` para que `proxy.ts` limpie las cookies del servidor.

DropdownMenu: Componente de `shadcn` basado en `Radix UI`. Es accesible (navegable con teclado), se cierra al hacer clic fuera, y se posiciona automáticamente para no salir de la pantalla.

text-destructive: Clase CSS de `shadcn` que aplica el color rojo del tema al botón de `logout`. Convención visual para acciones peligrosas.

Integrar el Header en el layout del dashboard

Ahora modificamos el layout del dashboard para usar el nuevo componente Header y pasarle los datos del usuario.

Actualiza `app/(dashboard)/layout.tsx` reemplazando todo su contenido:

```

Archivo: app/(dashboard)/layout.tsx (actualizado)
import { redirect } from "next/navigation"

```

```

import { createClient } from "@lib/supabase/server"

import {
  SidebarProvider,
  SidebarInset,
  SidebarTrigger,
} from "@components/ui/sidebar"
import { Separator } from "@components/ui/separator"
import { AppSidebar } from "@components/dashboard/app-sidebar"
import { DashboardHeader } from "@components/dashboard/header"

export default async function DashboardLayout({
  children,
}): {
  children: React.ReactNode
}) {
  const supabase = await createClient()

  const {
    data: { user },
  } = await supabase.auth.getUser()

  if (!user) {
    redirect("/login")
  }

  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("nombre_completo, rol")
    .eq("id", user.id)
    .single()

  if (!perfil) {
    redirect("/login")
  }

  return (
    <SidebarProvider>
      <AppSidebar
        rol={perfil.rol}
        nombreUsuario={perfil.nombre_completo}
      />
      <SidebarInset>
        {/* — Header mejorado — */}
        <header className="flex h-16 shrink-0 items-center gap-2 border-b px-4">
          <SidebarTrigger className="-ml-1" />
          <Separator
            orientation="vertical"
            className="mr-2 data-[orientation=vertical]:h-4"
          />
          <span className="text-sm font-medium">

```

```

        {perfil.rol.charAt(0).toUpperCase() + perfil.rol.slice(1)}
      </span>

      { /* Header con dropdown se posiciona a la derecha (ml-auto) */ }
      <DashboardHeader
        nombreUsuario={perfil.nombre_completo}
        email={user.email || ""}
        rol={perfil.rol}
      />
    </header>

    { /* — Contenido principal — */ }
    <main className="flex-1 p-6">
      {children}
    </main>
  </SidebarInset>
</SidebarProvider>
)
}

```

Tip: Composición Server + Client

Observa el patrón: el layout (Server Component) obtiene los datos con `await` y se los pasa al Header (Client Component) como props. El Header puede usar `useState`, `useRouter` y eventos porque es "use client". Cada componente hace lo que mejor sabe hacer.

Crear la API Route para actualizar el perfil

Esta API Route permite al usuario actualizar sus datos personales (nombre, teléfono, carrera). La usaremos en la Entrega 4A cuando construyamos la página de perfil.

Crea el archivo **app/api/auth/perfil/route.ts**:

```

Archivo: app/api/auth/perfil/route.ts
import { NextResponse } from "next/server"
import { createClient } from "@lib/supabase/server"

export async function PUT(request: Request) {
  try {
    const supabase = await createClient()

    // 1. Verificar que el usuario está autenticado
    const {
      data: { user },
    } = await supabase.auth.getUser()

    if (!user) {
      return NextResponse.json(
        { error: "No autorizado" },
        { status: 401 }
      )
    }
  }
}

```



```

    )
  }

  // 2. Leer el body de la petición
  const body = await request.json()
  const { nombre_completo, telefono, carrera } = body

  // 3. Validar campos obligatorios
  if (!nombre_completo || nombre_completo.trim().length < 2) {
    return NextResponse.json(
      { error: "El nombre debe tener al menos 2 caracteres" },
      { status: 400 }
    )
  }

  // 4. Actualizar el perfil
  const { data, error } = await supabase
    .from("usuarios_perfil")
    .update({
      nombre_completo: nombre_completo.trim(),
      telefono: telefono?.trim() || null,
      carrera: carrera?.trim() || null,
    })
    .eq("id", user.id)
    .select()
    .single()

  if (error) {
    return NextResponse.json(
      { error: "Error al actualizar: " + error.message },
      { status: 500 }
    )
  }

  return NextResponse.json({ perfil: data })
} catch {
  return NextResponse.json(
    { error: "Error interno del servidor" },
    { status: 500 }
  )
}
}

export async function GET() {
  try {
    const supabase = await createClient()

    const {
      data: { user },
    } = await supabase.auth.getUser()
  }

```

```

    if (!user) {
      return NextResponse.json(
        { error: "No autorizado" },
        { status: 401 }
      )
    }

    const { data: perfil, error } = await supabase
      .from("usuarios_perfil")
      .select("*")
      .eq("id", user.id)
      .single()

    if (error) {
      return NextResponse.json(
        { error: "Perfil no encontrado" },
        { status: 404 }
      )
    }

    return NextResponse.json({ perfil })
  } catch {
    return NextResponse.json(
      { error: "Error interno del servidor" },
      { status: 500 }
    )
  }
}

```

Aspectos clave de la API de perfil

Autenticación implícita: Usamos `createClient()` de `lib/supabase/server` (NO el `admin`). Esto respeta las políticas RLS. El usuario solo puede leer/editar SU propio perfil porque el `.eq("id", user.id)` coincide con la sesión actual.

PUT para actualizar: Seguimos la convención REST: PUT actualiza un recurso existente. El método `.select().single()` retorna el registro actualizado para confirmar los cambios.

GET para consultar: Permite obtener el perfil completo. Lo usaremos en la página de perfil para prellenar el formulario de edición.

Sanitización: Usamos `.trim()` para eliminar espacios en blanco y `|| null` para campos opcionales vacíos. Así evitamos guardar strings vacíos en la BD.

Crear las páginas placeholder restantes

Para que la navegación del sidebar funcione sin errores 404, creamos las páginas faltantes. Todas son Server Components simples.

Crea `app/(dashboard)/estudiante/materias/page.tsx`:

```
app/(dashboard)/estudiante/materias/page.tsx
export default function MateriasEstudiante() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Materias Disponibles</h1>
      <p className="text-muted-foreground">
        Aquí verás el catálogo de materias disponibles para inscripción.
      </p>
    </div>
  )
}
```

Crea `app/(dashboard)/estudiante/inscripciones/page.tsx`:

```
app/(dashboard)/estudiante/inscripciones/page.tsx
export default function InscripcionesEstudiante() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Mis Inscripciones</h1>
      <p className="text-muted-foreground">
        Aquí verás las materias en las que estás inscrito.
      </p>
    </div>
  )
}
```

Crea `app/(dashboard)/estudiante/perfil/page.tsx`:

```
app/(dashboard)/estudiante/perfil/page.tsx
export default function PerfilEstudiante() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Mi Perfil</h1>
      <p className="text-muted-foreground">
        Aquí podrás editar tu información personal.
      </p>
    </div>
  )
}
```

Crea `app/(dashboard)/docente/materias/page.tsx`:

```
app/(dashboard)/docente/materias/page.tsx
export default function MateriasDocente() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Mis Materias</h1>
      <p className="text-muted-foreground">
        Aquí verás las materias que tienes asignadas.
      </p>
    </div>
  )
}
```

```

    </p>
  </div>
)
}

```

Crea `app/(dashboard)/docente/perfil/page.tsx`:

```

app/(dashboard)/docente/perfil/page.tsx
export default function PerfilDocente() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Mi Perfil</h1>
      <p className="text-muted-foreground">
        Aquí podrás editar tu información personal.
      </p>
    </div>
  )
}

```

Crea `app/(dashboard)/admin/usuarios/page.tsx`:

```

app/(dashboard)/admin/usuarios/page.tsx
export default function UsuariosAdmin() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Gestión de Usuarios</h1>
      <p className="text-muted-foreground">
        Aquí podrás administrar todos los usuarios del sistema.
      </p>
    </div>
  )
}

```

Crea `app/(dashboard)/admin/materias/page.tsx`:

```

app/(dashboard)/admin/materias/page.tsx
export default function MateriasAdmin() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Gestión de Materias</h1>
      <p className="text-muted-foreground">
        Aquí podrás crear, editar y administrar las materias del sistema.
      </p>
    </div>
  )
}

```

Crea `app/(dashboard)/admin/inscripciones/page.tsx`:

```

app/(dashboard)/admin/inscripciones/page.tsx
export default function InscripcionesAdmin() {

```

```

return (
  <div className="space-y-4">
    <h1 className="text-3xl font-bold">Gestión de Inscripciones</h1>
    <p className="text-muted-foreground">
      Aquí podrás ver y gestionar todas las inscripciones del sistema.
    </p>
  </div>
)
}

```

Probar el flujo completo de la Fase 1

Con esta entrega completamos toda la Fase 1. Vamos a probar el flujo de principio a fin:

1. Ejecuta **npm run dev**
2. Visita <http://localhost:3000> — deberías ser redirigido a /login
3. Inicia sesión con tu usuario de prueba
4. Verifica el dashboard completo:
 - Sidebar con items según tu rol
 - Header con botón hamburguesa, nombre del rol, y avatar a la derecha
 - Contenido principal con el texto del placeholder
5. Prueba el dropdown de usuario:
 - Haz clic en el avatar circular (iniciales)
 - Deberías ver: nombre, email, rol
 - Haz clic en **Mi Perfil** — navega a la página de perfil
6. Prueba la navegación completa:
 - Haz clic en cada item del sidebar
 - Cada página debe mostrar su placeholder correspondiente
 - El item activo debe estar resaltado en el sidebar
7. **Prueba el logout:**
 - Haz clic en el avatar > **Cerrar Sesión**
 - Deberías ser redirigido a /login
 - Intenta navegar a /estudiante manualmente — deberías ser redirigido a /login
8. Prueba con otros roles:
 - Inicia sesión con un docente y verifica que ve items diferentes en el sidebar

- Inicia sesión con un admin y verifica que ve más items (Usuarios, Materias, Inscripciones, Configuración)

Tip: Prueba la API de perfil

Abre una nueva pestaña y visita <http://localhost:3000/api/auth/perfil>. Deberías ver el JSON con los datos de tu perfil (si estás autenticado) o un error 401 (si no lo estás).

Verificación Final — Cierre de Fase 1

Esta entrega cierra la Fase 1 (Fundación). Antes de avanzar a la Fase 2, verifica TODOS estos puntos:

Infraestructura

- ✓ Proyecto Next.js 16 con TypeScript, Tailwind y shadcn/ui funcionando
- ✓ Variables de entorno configuradas en .env.local
- ✓ 5 tablas creadas en Supabase (usuarios_perfil, materias, inscripciones, calificaciones, documentos_academicos)

Autenticación

- ✓ Registro con email/password funcional (crea usuario + perfil)
- ✓ Login funcional con redirección por rol
- ✓ Logout elimina la sesión y redirige a /login
- ✓ proxy.ts protege rutas — sin sesión redirige a /login
- ✓ Callback de confirmación de email funcional

Layout y navegación

- ✓ Sidebar dinámico que cambia items según el rol
- ✓ Sidebar colapsable (Ctrl+B o botón)
- ✓ Sidebar responsivo (panel deslizable en móvil)
- ✓ Header con avatar, dropdown de usuario y logout
- ✓ Páginas placeholder para TODAS las secciones (sin 404)

APIs

- ✓ POST /api/auth/registro — crea usuario + perfil
- ✓ GET /api/auth/perfil — retorna el perfil del usuario autenticado

✓ PUT /api/auth/perfil — actualiza nombre, teléfono, carrera

Ejercicio Propuesto

9. **Agregar foto al avatar:** Investiga el componente AvatarImage de shadcn. Modifica el header para que muestre la foto de avatar_url si existe, y las iniciales como fallback.
10. **Toast en logout:** Importa toast de sonner y muestra una notificación "Sesión cerrada exitosamente" antes de redirigir al login.
11. **Probar la API con fetch:** Abre la consola del navegador (F12 > Console) y ejecuta: `fetch("/api/auth/perfil").then(r => r.json()).then(console.log)`. ¿Qué datos ves?
12. **Reflexión arquitectónica:** Hemos construido la autenticación y la navegación. Dibuja en un papel el flujo completo: desde que el usuario abre el navegador hasta que ve su dashboard. ¿Cuántos archivos intervienen? ¿En qué orden se ejecutan?

Resumen de la Fase 1

Hemos construido la base completa de la aplicación en 6 entregas parciales. Este es el inventario de todo lo que creamos:

Entrega	Archivos creados	Funcionalidad
1A	next.config.ts, .env.local, page.tsx	Proyecto + Supabase Cloud
1B	globals.css, utils.ts, esquema SQL, components/ui/*	shadcn + BD + estructura
2A	client.ts, server.ts, proxy.ts, callback/route.ts	Infraestructura de auth
2B	login-form, registro-form, /api/auth/registro	Login + registro funcional
3A	navigation.ts, app-sidebar, layout dashboard, placeholders	Sidebar dinámico por rol
3B	header.tsx, use-usuario.ts, /api/auth/perfil, placeholders	Header + logout + API perfil

Próximo: Fase 2 — Gestión de Usuarios y Materias

En la Fase 2 reemplazaremos los placeholders con funcionalidad real. La Entrega 4A creará la página de perfil con formulario editable usando React Hook Form + Zod, y la Entrega 4B construirá el panel de administración de usuarios.